

Analisi di moloch.F90

Modello non-idrostatico MOLOCH — versione 25.2.2

repository `globo-bolam-moloch-at-lamma`

Struttura del documento

Parte I	Analisi delle routine	pag. 2
Parte II	Report dei bug con fix proposti	pag. 15

File analizzato	<code>sources/moloch/moloch.F90</code>	
Righe totali	15 119	
Routine totali	66	(subroutine + function)
Bug confermati		
Critici	1	
Alti	7	
Medi	2	

Indice

Parte I — Analisi delle Routine	2
Introduzione	2
1 Parallelizzazione MPI	3
1.1 u_ghost_init — riga 301	3
1.2 u_ghost_put_2d / u_ghost_put_1d / u_ghost_put_0d — righe 330, 382, 434	3
1.3 u_ghost_transfer — riga 484	3
1.4 u_ghost_get_2d / u_ghost_get_1d / u_ghost_get_0d — righe 560, 600, 640	3
1.5 u_ghost_old — riga 7623	3
1.6 disperse / disperse_int — righe 3314, 3359	3
1.7 collect / collect_int — righe 4599, 4656	4
2 Dinamica e Schema Numerico	5
2.1 Coordinate verticali — righe 246–258	5
2.2 paidef — riga 10897	5
2.3 Schema di avvezione WAF — righe 8370–9454	5
2.3.1 advect_waf — riga 8370	5
2.3.2 wafone — riga 9455	5
2.3.3 rdeno — riga 240	5
2.4 Filtri orizzontali — righe 3560–3694	6
2.4.1 filt2d — riga 3560	6
2.4.2 filt3d — riga 3695	6
2.4.3 aver — riga 3627	6
2.4.4 lpfilts — riga 12813	6
2.5 Diffusione orizzontale — righe 11085–11321	6
2.5.1 hdiffu — riga 11085	6
2.5.2 hdiff_t — riga 11168	6
2.6 Smorzatori — righe 12712–12812	6
2.6.1 divdamp — riga 12712	6
2.6.2 relax — riga 3766	7
2.6.3 nudging_tuv — riga 12770	7
2.7 Interpolazione — riga 10926	7
2.7.1 interp — riga 10926	7
2.8 Utilità geometriche	7
2.8.1 raglio — riga 12960	7
2.8.2 calendar — riga 10764	7
3 Parametrizzazioni Fisiche	8
3.1 surflayer — riga 4924	8
3.2 vdiff — riga 5477	8
3.3 tofd — riga 5196	8
3.4 orogdrag — riga 5221	8
3.5 cloudfr — riga 5804	8
3.6 ccloudt — riga 5842	8
3.7 Schema radiativo — righe 5914–7622	9
3.7.1 radiat_init — riga 5914	9
3.7.2 radint — riga 6187	9
3.7.3 radial — riga 6422	9

3.7.4	radintec — riga 10206	9
3.7.5	aerdef — riga 9704	9
3.8	convection_kf — riga 11540	9
3.8.1	hliq — riga 12625	9
3.9	sea_surface — riga 10845	10
3.10	surfradpar — riga 11322	10
3.11	pbl_height — riga 11501	10
3.12	snowfall_level — riga 12914	10
3.13	comp_esk — riga 9668	10
4	Microfisica	11
4.1	micro2m — riga 8628	11
4.2	eugamma — riga 9415	11
4.3	plotout — riga 11060	11
5	Input/Output	12
5.1	Lettura condizioni iniziali e al contorno	12
5.1.1	rdmhf_atm — riga 2721	12
5.1.2	rdmhf_soil — riga 2867	12
5.1.3	rd_param_const — riga 3109	12
5.2	Scrittura output atmosferico	12
5.2.1	wrmhf_atm / wrmhf_atm_full_res — righe 3838, 4309	12
5.2.2	wrmhf_soil / wrmhf_soil_full_res — righe 3923, 4388	12
5.2.3	wr_param_const — riga 4140	12
5.2.4	wrshf — riga 7654	12
5.2.5	wrshf_radar — riga 8248	13
5.3	Restart	13
5.3.1	rdrf — riga 13249	13
5.3.2	wrrf — riga 13726	13
5.3.3	RDRF_POCHVA / WRRF_POCHVA — righe 14169, 14861	13
5.4	Output diagnostico	13
5.4.1	runout — riga 4712	13
5.4.2	wrspray — riga 13010	13
5.4.3	wrback — riga 11446	13
5.5	Utilità I/O di basso livello	14
Parte II	Report dei Bug con Fix Proposti	15
	Riepilogo	15
6	Bug #1 — jp1 sempre uguale a 1 (staggering COSMO)	16
7	Bug #2 — Bound errato per ip1 (staggering COSMO)	17
8	Bug #3 — Tipo MPI errato in disperse_int	18
9	Bug #4 — Divisione per zero nella convergenza Holtslag	19
10	Bug #5 — zprod non inizializzato in ccloudt	20
11	Bug #6 — Indice errato in grib2_descript (wrshf_radar)	21
12	Bug #7 — llfc non inizializzato in convection_kf	22

13 Bug #8 — Divisione per zero nel calcolo dell'umidità relativa del suolo	23
14 Bug #9 — Confronto con dimensione errata in raglio	24
15 Bug #10 — Troncamento intero silente in nfdrr	25
Conclusioni	26

Parte I — Analisi delle Routine

Introduzione

moloch.F90 contiene l'intero nucleo del modello mesoscala MOLOCH (versione *theta-pai*, non-idrostatica). Il file è organizzato come un programma Fortran 90 con un blocco principale (`program moloch`) seguito da 66 subroutine e funzioni raggruppabili in sei famiglie funzionali:

Categoria	Descrizione sintetica	N. routine
MPI	Ghost cells, scatter, gather	13
Dinamica/Num.	Coordinata, avvezione, filtri, diffusione	16
Fisica	Surface layer, PBL, radiazione, convezione	19
Microfisica	Schema bulk 2-momenti	3
I/O	Lettura/scrittura MHF, SHF, restart	21
Diagnostica	Output runtime, plot	2

Parallelizzazione MPI

Il modello supporta la decomposizione di dominio su griglia MPI 2D (`nprocsx`×`nprocsy` processi). Ogni processo possiede una sotto-griglia con *ghost cells* (celle fantasma) di bordo che vengono scambiate prima di ogni operazione che richiede valori adiacenti. A partire dalla versione 25 (Feb. 2025) l'infrastruttura è stata riscritta per raggruppare più scambi in un singolo buffer collettivo, riducendo il numero di chiamate MPI.

`u_ghost_init` — riga 301

Inizializza il buffer di scambio interno e il comunicatore MPI (`use_comm`) che verrà usato da tutte le routine `u_ghost_*`. Va chiamata *una volta* all'inizio di ogni fase di scambio collettivo, prima di qualsiasi `u_ghost_put_*`.

`u_ghost_put_2d` / `u_ghost_put_1d` / `u_ghost_put_0d` — righe 330, 382, 434

Caricano nel buffer di invio, rispettivamente, una slice 2D, un vettore 1D o uno scalare. Il parametro `idirect` codifica la direzione: 1=Nord, 2=Est, 3=Sud, 4=Ovest. Più campi possono essere impilati sullo stesso buffer prima dell'invio effettivo, riducendo la latenza MPI.

`u_ghost_transfer` — riga 484

Esegue l'effettivo scambio non-bloccante dei buffer tra i processi vicini nelle quattro direzioni (`ip_n`, `ip_s`, `ip_e`, `ip_w`). Internamente usa `MPI_Isend`/`MPI_Irecv` seguiti da `MPI_Waitall`. È il punto di sincronizzazione della comunicazione.

`u_ghost_get_2d` / `u_ghost_get_1d` / `u_ghost_get_0d` — righe 560, 600, 640

Estraggono dal buffer di ricezione i dati ricevuti dai processi vicini e li copiano nelle ghost cells degli array locali. Vanno chiamate dopo `u_ghost_transfer`.

`u_ghost_old` — riga 7623

Versione *legacy* dello scambio ghost cells: gestisce una singola coppia send/recv sincrona. Presente nel codice per compatibilità con sezioni non ancora migrate al nuovo schema; è commentata nella maggior parte dei punti di chiamata.

`disperse` / `disperse_int` — righe 3314, 3359

Il processo root (`myid=0`) legge un campo globale da file e lo distribuisce (`MPI_Send` → `MPI_Recv`) ai singoli processi che ricevono la propria sotto-griglia locale. `disperse` gestisce array `real`; `disperse_int` gestisce array `integer` (contiene il bug MPI #3, vedi Parte II).

`collect` / `collect_int` — righe 4599, 4656

Operazione inversa di **disperse**: ogni processo invia la propria sotto-griglia al root che ricostruisce il campo globale per la scrittura su file. `collect_int` è l'analogia per interi.

Dinamica e Schema Numerico

Coordinate verticali — righe 246–258

MOLOCH usa la coordinata verticale ibrida *zita* ($\zeta \in [0, 1]$) che segue il terreno vicino alla superficie e si inclina verso l'orizzontale in quota secondo la funzione di stretching $b(\zeta)$ e quella di decadimento $g(\zeta)$.

Funzione	Riga	Descrizione
<code>gzita</code>	246	Funzione di decadimento $g(\zeta)$
<code>gzitap</code>	250	Derivata $g'(\zeta)$
<code>bzita</code>	254	Funzione di stretching $b(\zeta)$
<code>bzitap</code>	258	Derivata $b'(\zeta)$

Queste funzioni sono utilizzate ovunque si calcoli l'altezza reale z a partire da ζ : $z(\zeta) = -h \cdot b(\zeta) \cdot \ln(1 - \zeta/h)$.

paidef — riga 10897

Calcola la funzione di Exner $\Pi = (p/p_0)^{R_d/c_{pd}}$ su tutti i livelli del profilo verticale passato. Usata per convertire tra temperatura potenziale θ e temperatura assoluta T . Input: p , T , q , q_{cw} , q_{ci} . Output: profilo di Π .

Schema di avvezione WAF — righe 8370–9454

Lo schema WAF (*Weighted Average Flux*) è il nucleo dinamico per l'avvezione di tutti i campi prognostici.

advect_waf — riga 8370

Orchestratore dell'avvezione 3D. Per ogni variabile prognostica (`tetav`, `pai`, `q`, `qcw`, `qci`, `qpw`, `qpi1`, `qpi2`, `tke`, `u`, `v`, `w`) chiama in sequenza:

1. scambio ghost cells MPI (orizzontale);
2. interpolazione del vento sui punti scalari;
3. chiamata a `wafone` per avvezione 1D in ciascuna direzione.

Il flusso numerico WAF è ottenuto come media pesata dei flussi Upwind e Lax-Wendroff, con limitatore di flusso per evitare oscillazioni.

wafone — riga 9455

Esegue un singolo passo WAF 1D su un vettore di campo e il corrispondente vettore di velocità. Applica il limitatore *superbee* per la correzione di flusso. Opzionalmente (`nltwice`) esegue il passo in due semi-passi temporali per maggiore stabilità (v. Ago. 2019).

rdeno — riga 240

Funzione ausiliaria interna allo schema WAF: calcola il rapporto di smoothness r tra il gradiente upstream e il gradiente locale, usato dal limitatore di flusso.

Filtri orizzontali — righe 3560–3694**filt2d — riga 3560**

Applica un filtro di Shapiro del quarto ordine (∇^4) a un campo 2D $p(nlon, nlat)$. Il parametro **anu2** controlla l'intensità: **anu2** = 1/16 corrisponde al massimo smorzamento della lunghezza d'onda di Nyquist. Scambia i ghost cells MPI prima del calcolo per garantire la coerenza ai bordi del sotto-dominio.

filt3d — riga 3695

Estensione 3D di **filt2d**: applica il filtro Shapiro su ogni livello verticale del campo $p(nlon, nlat, nlev)$.

aver — riga 3627

Media pesata del campo p su una griglia sfalsata (staggered): sostituisce ogni valore con la media aritmetica dei quattro punti adiacenti. Usata per lisciare campi orografici o di pressione.

lpfilts — riga 12813

Filtro passa-basso spettrale applicato ai campi prima del nudging. Elimina le scale sub-filter per evitare che il nudging introduca rumore ad alta frequenza dove i dati di bordo sono a risoluzione più bassa di MOLOCH.

Diffusione orizzontale — righe 11085–11321**hdiffu — riga 11085**

Diffusione orizzontale ∇^4 dei campi di vento (u, v, w). Usa un coefficiente di diffusione che decresce con la quota (per evitare l'over-damping in alta troposfera). Aggiorna le tendenze **ut**, **vt**, **wt** sommando il contributo diffusivo.

hdifft — riga 11168

Diffusione orizzontale di temperatura virtuale potenziale (**tetav**), umidità specifica (**q**) e delle specie idrometeoriche. Come **hdiffu**, aggiorna le tendenze corrispondenti.

Smorzatori — righe 12712–12812**divdamp — riga 12712**

Aggiunge un termine di smorzamento proporzionale alla divergenza orizzontale $D = \partial u / \partial x + \partial v / \partial y$. Controlla le oscillazioni acustico-gravitazionali nel modello non-idrostatico. Introdotto a Dic. 2020.

relax — riga 3766

Rilassamento lineare (*Davies relaxation*) ai bordi laterali: in una banda di larghezza configurabile i campi prognostici vengono progressivamente riportati verso i valori delle condizioni al contorno. Il coefficiente di rilassamento varia da 0 (interno) a 1 (bordo esterno).

nudging_tuv — riga 12770

Nudging spettrale di T , U , V verso i dati di input su tutto il dominio (non solo ai bordi). Applicato dopo il filtro passa-basso `lpfilt5`. Il coefficiente di nudging è letto da `namelist`. Introdotto a Ott. 2021.

Interpolazione — riga 10926**interp — riga 10926**

Interpolazione spline cubica monotona 1D. Dato un vettore di coordinate ξ (non necessariamente uniformi) e valori $g(\xi)$, calcola i valori interpolati nei punti x richiesti. Usata per l'interpolazione verticale (da livelli sigma/zita a livelli di pressione standard per l'output). Risolve localmente i problemi di over/undershoot tramite monotonizzazione.

Utilità geometriche**raglio — riga 12960**

Data una posizione geografica (lon, lat) e la griglia globale, restituisce gli indici ($i1, j1$) del sotto-dominio $n_x \times n_y$ centrato su quel punto. Usata da `wrspray` per localizzare la finestra di output dello spray marino.

calendar — riga 10764

Converte un numero di giorni da una data di riferimento in anno/mese/giorno/ora/minuto. Gestisce correttamente gli anni bisestili. Usata per calcolare la declinazione solare e nell'output temporale.

Parametrizzazioni Fisiche

surflayer — riga 4924

Strato superficiale (Monin-Obukhov). Calcola i flussi turbolenti di superficie: quantità di moto (τ), calore sensibile (H), vapore (E). Implementa le funzioni di stabilità di *Businger* (caso instabile) e *Holtslag* (caso stabile) tramite loop iterativo di convergenza. Output principali: `ustar`, `tstar`, `qstar`, `hflux`, `qflux`, coefficienti di scambio `roscdm`, `roscdt`. Calcola anche vento a 10 m, temperatura e umidità a 2 m per l'interpolazione diagnostica.

vdifff — riga 5477

Diffusione verticale turbolenta (chiusura E-L). Calcola la lunghezza di mescolamento (Blackadar o Deardorff), la TKE (*Turbulent Kinetic Energy*), e integra implicitamente la diffusione verticale di u , v , θ_v , q , q_{cw} , q_{ci} . Schema implicito: risolve un sistema tridimensionale per ogni colonna. Versione *theta-pai* (v17): lavora in coordinate (θ, Π) invece di (T, p) per maggiore consistenza termodinamica.

tofd — riga 5196

Trascinamento orografico turbolento (TOFD). Parametrizza la resistenza aggiuntiva esercitata sullo strato limite da rilievi sub-griglia (colline, valli non risolte dalla griglia). Aggiunge un termine di drag a u e v proporzionale alla varianza dell'orografia sub-griglia. Introdotto a Gen. 2019.

orogdrag — riga 5221

Drag orografico e blocking da onde di gravità. Calcola il flusso di momento verticale generato dalle onde orografiche e il *blocking* dello strato limite da rilievi alti. Segue lo schema di *Palmer et al.* per la propagazione verticale delle onde e il criterio di criticità di Richardson per il wave-breaking.

cloudfr — riga 5804

Frazione nuvolosa diagnostica. Calcola la copertura nuvolosa parziale `cloudf(jlon, jlat, jklev)` come funzione della umidità relativa: nullo sotto la soglia RH_{crit} , unitario quando $RH = 1$. Usata dallo schema radiativo per calcolare la trasmittanza di strato.

ccloudt — riga 5842

Copertura nuvolosa totale (integrata sulla colonna). Combina le frazioni nuvolose dei singoli strati con la regola *Maximum-Random Overlap*: la copertura massima tra strati adiacenti è usata per strati contigui; la sovrapposizione casuale per strati separati da cielo chiaro. Output: `cloudt(jlon, jlat)`. **Contiene il bug #5 (zprod non inizializzato), vedi Parte II.**

Schema radiativo — righe 5914–7622

radiat_init — riga 5914

Inizializza le tabelle di assorbimento dei gas serra (H_2O , CO_2 , O_3), le proprietà ottiche degli aerosol e la distribuzione di ozono. Chiamata una sola volta all'avvio del modello.

radint — riga 6187

Orchestratore della radiazione: calcola l'angolo zenitale solare, corregge per la pendenza del terreno (`slopeff`), chiama `radial` per ogni latitudine. Aggiorna i flussi radiativi `rds` (SW in), `ret` (LW up), `rls` (LW down). Chiamata a ogni passo di radiazione (ogni `nstep_rad` passi).

radial — riga 6422

Calcolo radiativo completo per ogni colonna: onda corta (SW) e onda lunga (LW). Implementa gli schemi a due-stream (*delta-Eddington* per SW, *emissività* per LW). Effettua la trasformazione di coordinate da zita a livelli di pressione standard (`pap5`) per la compatibilità con le tabelle di assorbimento.

radintec — riga 10206

Schema radiativo alternativo ECMWF (compilato con `#ifdef rad_ecmwf`). Usa le tabelle e la parametrizzazione del Centro Europeo. Richiede il package esterno ECMWF opzionale.

aerdef — riga 9704

Definisce la distribuzione verticale e spettrale degli aerosol in funzione del mese e della posizione geografica. Usata da `radiat_init`.

convection_kf — riga 11540

Schema convettivo Kain-Fritsch. Parametrizza la convezione profonda e superficiale. Calcola il livello di sollevamento libero (LFC), il livello di galleggiamento neutro (EL), il flusso di massa convettivo con profilo a cappello (*top-hat*), le tendenze di T , q , q_{cw} dovute alla convezione e la precipitazione convettiva. Il parametro `timec` controlla il tempo di rilassamento del CAPE (introdotto a Set. 2020). **Contiene il bug #8 (llfc non inizializzato), vedi Parte II.**

hliq — riga 12625

Calcola i cambiamenti di fase (condensazione/evaporazione/fusione/ congelamento) all'interno del profilo convettivo Kain-Fritsch. Aggiorna la temperatura tenendo conto del calore latente.

sea_surface — riga 10845

Aggiorna la SST (*Sea Surface Temperature*) interpolando linearmente nel tempo tra i campi MHF disponibili. Applica la maschera terra/mare **fmask** e gestisce il ghiaccio marino.

surfradpar — riga 11322

Calcola i parametri radiativi della superficie: albedo, emissività in funzione del tipo di vegetazione, della copertura nevosa e dell'umidità del suolo. Corrected May 2022 per la frazione di vegetazione.

pbl_height — riga 11501

Stima l'altezza dello strato limite planetario cercando il primo livello in cui il numero di Richardson bulk supera la soglia critica $Ri_c = 0.25$. Usata per diagnostica e per la convezione superficiale.

snowfall_level — riga 12914

Determina il livello di neve al suolo: risale la colonna dal basso fino al livello dove la precipitazione è prevalentemente liquida (temperatura $> 0^{\circ}\text{C}$). Usata nell'output MHF del suolo.

comp_esk — riga 9668

Calcola la pressione di vapore di saturazione e_{sat} (formula di Magnus o Tetens) e l'umidità specifica di saturazione q_{sat} sia rispetto all'acqua liquida (**iflag=1**) che al ghiaccio (**iflag=2**). Usata ovunque nel modello.

Microfisica

micro2m — riga 8628

Schema microfisico bulk a 1 o 2 momenti.

Gestisce l'evoluzione di 5 categorie di idrometeorie: nube liquida (q_{cw} , N_{cw}), nube ghiacciata (q_{ci} , N_{ci}), pioggia (q_{pw}), neve/grandine tipo 1 (q_{pi1}), grandine tipo 2 (q_{pi2}). Con la flag `nlmic2=.true.` vengono evoluti anche i numeri N (2 momenti).

Parametrizzazione delle distribuzioni:

- Nube liquida/ghiacciata: distribuzione gamma con parametro $\alpha_{cw} = 6$, $\alpha_{ci} = 3$;
- Pioggia/neve/grandine: distribuzione esponenziale di Marshall-Palmer con N_0 fisso (1-momento) o variabile (2-momenti).

Processi parametrizzati (circa 20 in totale):

1. Nucleazione omogenea e eterogenea del ghiaccio
2. Condensazione/deposizione per diffusione vapore
3. Autoconversione nube→pioggia (Kessler) e nube ghiacciata→neve
4. Accrescimento per collisione: pioggia+nube, neve+nube, grandine+nube
5. Accrescimento per collisione: pioggia+neve → grandine
6. Evaporazione di pioggia e sublimazione di neve/grandine
7. Fusione neve/grandine sotto lo zero
8. Congelamento di gocce supercooled (Bigg)
9. Sedimentazione verticale con flussi upwind
10. Calcolo riflettività radar simulata (`nlradar=.true.`)

Struttura: Loop esterno su latitudini (`jlat`), loop interno su longitudini (`jlon`), poi loop verticale *dall'alto verso il basso* per includere la sedimentazione. Output: tendenze `dqcwdt`, `dqcidt`, `dqpwdt`, `dqpi1dt`, precipitazione `raini`, `snowi`.

eugamma — riga 9415

Calcola la funzione Gamma di Eulero $\Gamma(x)$ per argomenti reali positivi tramite l'approssimazione di Lanczos. Usata da `micro2m` per calcolare i momenti delle distribuzioni gamma (es. media della velocità di caduta $\propto \Gamma(\alpha + b + 1)$).

plotout — riga 11060

Stampa ASCII di una matrice 2D (per debug): produce un'immagine testuale a simboli del campo normalizzato. Non usata in produzione.

Input/Output

Lettura condizioni iniziali e al contorno

`rdmhf_atm` — riga 2721

Lettura file MHF atmosfera. Legge un file binario unformatted nel formato MHF (MOLOCH History File) contenente i campi atmosferici 3D: u , v , w , θ_v , Π , q , q_{cw} , q_{ci} , q_{pw} , q_{pi1} , q_{pi2} , TKE. Il parametro `init_flag=1` indica lettura all'istante iniziale (allocazione e interpolazione dalla griglia BOLAM a quella MOLOCH); `init_flag=0` indica aggiornamento dei bordi laterali. Il formato dal Giu. 2024 usa record 2D invece di 1D.

`rdmhf_soil` — riga 2867

Lettura file MHF suolo. Legge i campi del suolo: temperatura a più livelli (`tsoil`), umidità a più livelli (`qg`), manto nevoso multi-strato (`snow`, 11 livelli da v. 2018). Include lettura di `model_param_constant.bin` tramite `rd_param_const` al passo iniziale.

`rd_param_const` — riga 3109

Legge il file `model_param_constant.bin` (formato aggiornato a Sett. 2024) contenente i campi fisiografici costanti: orografia (`phig`), maschera terra/mare (`fmask`), mappa di vegetazione, parametri idraulici del suolo (`qsoil_max`, `qsoil_min`). Verifica la consistenza dell'header con i parametri del modello tramite la variabile `ierr`.

Scrittura output atmosferico

`wrmhf_atm` / `wrmhf_atm_full_res` — righe 3838, 4309

Scrive il file MHF atmosferico per il nesting: `wrmhf_atm` scrive alla risoluzione ridotta (decimata di un fattore `mhfr`); `wrmhf_atm_full_res` scrive a risoluzione piena. Usate quando MOLOCH funge da modello padre per un run nidificato.

`wrmhf_soil` / `wrmhf_soil_full_res` — righe 3923, 4388

Scrittura analoga per i campi del suolo.

`wr_param_const` — riga 4140

Scrive `model_param_constant.bin` alla risoluzione decimata. Necessario per avviare un run MOLOCH nidificato in MOLOCH.

`wrshf` — riga 7654

Scrittura file SHF (history file principale). Scrive in formato GRIB2 i campi selezionati in output: campi 2D di superficie (precipitazione, flussi, pressione) e campi 3D su livelli verticali. Usa `collect` per raccogliere i dati da tutti i processi MPI prima della

scrittura (solo il root scrive).

wrshf_radar — riga 8248

Scrive in GRIB2 la riflettività radar simulata interpolata su 40 livelli a quota costante (0–11 km). **Contiene il bug #6 (grib2_descript(iunit)), vedi Parte II.**

Restart

rdrf — riga 13249

Legge il file di restart completo: tutti i campi prognostici (u , v , w , θ_v , Π , q , idrome-teore, TKE, suolo) da file binario unformatted. Ripristina anche il contatore del passo temporale. Ogni processo legge la propria porzione di dominio. Introdotto a Feb. 2023.

wrrf — riga 13726

Scrive il file di restart. Il root raccoglie i campi con `collect` e scrive l'intero dominio. Chiamato ogni `nstep_rf` passi.

RDRF_POCHVA / WRRF_POCHVA — righe 14169, 14861

Lettura e scrittura del restart per lo schema suolo-vegetazione POCHVA (Hydrology-POCHVa): gestisce i campi del suolo multicategoria e il manto nevoso multi-strato. Usa `disperse/collect` per la distribuzione parallela.

Output diagnostico

runout — riga 4712

Output diagnostico ad ogni passo: stampa su `stdout` il tempo, i valori estremi di u , v , w , p , T_{skin} e la posizione del minimo/massimo di pressione. In parallelo, raccoglie i valori locali da ogni processo tramite `MPI_Gather` prima di determinare il massimo globale.

wrspray — riga 13010

Scrive i campi di spray marino (droplet di sale) su una finestra geografica specificata, centrata sul punto di interesse. Usa `raglio` per calcolare gli indici della sotto-griglia.

wrback — riga 11446

Scrive lo stato completo del modello su file di backup (`backup.bin`) per la ripresa di un run interrotto. Distinto dal restart completo in quanto include anche variabili interne di diagnostica.

Utilità I/O di basso livello

Routine	Riga	Funzione
rrec2	3404	Lettura record 2D real (formato corrente)
rrec2_int	3416	Lettura record 2D integer
rrec2_old	3428	Lettura record 2D real (formato pre-2024)
rrec2_int_old	3442	Lettura record 2D integer (legacy)
wrec2	3456	Scrittura record 2D real
wrec2r	3470	Scrittura record 2D real (variante)
wrec2r_old	3488	Scrittura record 2D real (legacy)
wrec2_int	3508	Scrittura record 2D integer
wrec2r_int	3522	Scrittura record 2D integer (variante)
wrec2r_int_old	3540	Scrittura record 2D integer (legacy)

Queste routine incapsulano la lettura/scrittura di singoli record bidimensionali in file binari unformatted. Le varianti `_old` leggono il formato pre-Giu. 2024 (record 1D); le versioni correnti usano record 2D, eliminando il loop di lettura riga per riga. Le varianti `r` (`wrec2r`) scrivono senza avanzare il puntatore al record successivo (utile per aggiornamenti parziali).

Parte II — Report dei Bug con Fix Proposti

Riepilogo

#	Routine	Riga	Tipo	Severità
1	Programma principale	1381	Logica (min errato)	CRITICO
2	Programma principale	1383	Array out-of-bounds	ALTO
3	<code>disperse_int</code>	3387/3390	Tipo MPI errato	ALTO
4	<code>surflayer</code>	5028	Divisione per zero	ALTO
5	<code>ccloudt</code>	5903/5905	Variabile non inizializzata	ALTO
6	<code>wrshf_radar</code>	8336	Indice array errato	ALTO
7	<code>convection_kf</code>	11627	Variabile non inizializzata	ALTO
8	Programma principale	2200/2201	Divisione per zero	ALTO
9	<code>raglio</code>	13002	Indice dimensione errata	MEDIO
10	Programma principale	1364/1365	Troncamento intero silente	MEDIO

Bug #1 — jp1 sempre uguale a 1 (staggering COSMO)

Routine: programma principale, blocco interpolazione vento da input COSMO

Codice problematico:

```

1379 elseif (input_model == 'COSMO') then
1380   do j = 1, nlat_inp
1381     jp1 = min(1, j+1)      ! BUG: sempre 1
1382   do i = 1, nlon_inp
1383     ip1 = min(nlon, i+1)
1384     u_inp_work(i,j) = 0.5*(u_inp(i,j,k) + u_inp(ip1,j,k))
1385     v_inp_work(i,j) = 0.5*(v_inp(i,j,k) + v_inp(i,jp1,k)) ! jp1 = 1
                                sempre

```

Spiegazione: Il loop esterno parte da $j = 1$, quindi $j+1 \geq 2$ in ogni iterazione. `min(1, j+1)` restituisce sempre 1. La riga 1385 dovrebbe mediare $v_{inp}(i, j)$ con $v_{inp}(i, j+1)$ per implementare lo staggering della griglia Arakawa-C: invece usa sempre $v_{inp}(i, 1)$ — la prima riga della griglia. Ne consegue che `v_inp_work` è sistematicamente sbagliato per ogni $j > 1$, cioè per tutta la griglia eccetto la prima riga.

Impatto: Vento meridionale (v) delle condizioni iniziali/al contorno da COSMO completamente errato. Tutti i run MOLOCH con input COSMO producono campi iniziali incorretti.

Fix:

```

jp1 = min(nlat_inp, j+1)

```

Bug #2 — Bound errato per ip1 (staggering COSMO)

Routine: programma principale, stesso blocco del Bug #1

Codice problematico:

```
1382 do i = 1, nlon_inp
1383     ip1 = min(nlon, i+1)           ! BUG: nlon e' la dimensione MOLOCH
1384     u_inp_work(i,j) = 0.5*(u_inp(i,j,k) + u_inp(ip1,j,k))
```

Spiegazione: `nlon` è la dimensione longitudinale della griglia MOLOCH (output), mentre `u_inp` ha prima dimensione `nlon_inp` (griglia input COSMO). Tipicamente MOLOCH ha risoluzione più alta: `nlon > nlon_inp`. Quando `i` si avvicina a `nlon_inp`, `min(nlon, i+1)` restituisce `i+1` che può superare `nlon_inp`, causando accesso fuori bounds.

Impatto: Lettura di memoria arbitraria; corruzione silente del vento zonale sull'ultima colonna della griglia.

Fix:

```
1 ip1 = min(nlon_inp, i+1)
```

Bug #3 — Tipo MPI errato in disperse_int

Routine: disperse_int

Codice problematico:

```
3380 ! La subroutine gestisce array INTEGER (igfield, ilfield)
3381 if (myid == 0) then
3382   do jpr = 1, nprocsx*nprocsy-1
3383     ...
3384     call mpi_send (igfield(sx:ex,sy:ey), count, mpi_real, & ! BUG
3385                   jpr, tag, comm, error)
3386   enddo
3387 else
3388   call mpi_recv (ilfield, count, mpi_real, & ! BUG
3389                0, tag, comm, status, error)
3390 endif
```

Spiegazione: `disperse_int` è esplicitamente progettata per distribuire campi interi (come le maschere di tipo suolo). Tuttavia sia `mpi_send` che `mpi_recv` specificano `mpi_real` come tipo MPI, mentre il tipo corretto è `mpi_integer`. Su architetture dove `sizeof(integer) ≠ sizeof(real)` il numero di byte trasferiti è errato. Anche quando le dimensioni coincidono, la violazione della specifica MPI rende il comportamento non portabile e potenzialmente indefinito.

Impatto: Corruzione silente dei campi interi distribuiti in esecuzione parallela (maschere, indici di tipo suolo).

Fix:

```
1 call mpi_send (igfield(sx:ex,sy:ey), count, mpi_integer, jpr, tag, comm,
2               error)
3 ...
4 call mpi_recv (ilfield, count, mpi_integer, 0, tag, comm, status, error)
```

Bug #4 — Divisione per zero nella convergenza Holtslag

Routine: surflayer

Codice problematico:

```

5020 if(zri.gt.0.) then      ! funzioni di stabilita' Holtslag (caso stabile)
5021   zal = .2 + 4.*zri
5022   ...
5023   do jiter = 1, 10
5024     zalm = zal
5025     zal = zri*(zalmaz-zpsim)**2/(zalmaz-zpsih)  ! zal puo' = 0
5026     error = abs(zal-zalm)/zal                  ! BUG: /0 se zal=0
5027     if (error.lt.1.e-2) go to 1
5028     ...
5029   enddo

```

Spiegazione: Se (zalmaz-zpsim) == 0 durante le iterazioni, zal diventa zero e la riga successiva divide per zero. La condizione zri.gt.0. non esclude questa eventualità.

Impatto: FPE (*floating point exception*) con -ffpe-trap=zero; altrimenti Inf/NaN silente che si propaga nei coefficienti di scambio turbolento.

Fix:

```

1  if (zal /= 0.) then
2    error = abs(zal-zalm)/zal
3  else
4    error = 0.  ! convergenza immediata se zal=0
5  endif

```

Bug #5 — zprod non inizializzato in ccloudt

Routine: ccloudt

Codice problematico:

```
5896 pden = 1.  
5897 do k = 1, kmi  
5898     pden = pden*(1.-zcol(kmin(k)))  
5899 enddo  
5900  
5901 ! zprod viene assegnato SOLO se pden != 0  
5902 if (pden.ne.0.) zprod = pnum/pden  
5903  
5904 ! zprod usato SEMPRE, anche se pden=0  
5905 cloudt(jlon,jlat) = max (0., 1.-zprod)
```

Spiegazione: Se almeno un elemento `zcol(kmin(k))` vale esattamente 1.0, `pden` diventa zero e `zprod` non viene mai assegnato. La riga 5905 usa `zprod` incondizionatamente: conterrà il valore dell'iterazione precedente del loop su `(jlon, jlat)`, copiando la copertura nuvolosa totale da un punto di griglia adiacente.

Impatto: Copertura nuvolosa totale (`cloudt`) errata in corrispondenza di colonne completamente sature; il bug è silente e si manifesta solo in condizioni di saturazione totale (es. nebbia densa o convezione profonda organizzata).

Fix:

```
1 zprod = 0. ! copertura totale se pden=0  
2 if (pden.ne.0.) zprod = pnum/pden  
3 cloudt(jlon,jlat) = max (0., 1.-zprod)
```

Bug #6 — Indice errato in grib2_descriptor (wrshf_radar)

Routine: wrshf_radar

Codice problematico:

```

8260 integer, dimension(50) :: grib2_descriptor = 0
8261 ...
8262 integer :: iunit=41, ...      ! iunit = file unit number
8263
8264 ! Tutti gli altri usi usano indici fissi:
8265 grib2_descriptor( 3) = 0      ! instant product
8266 grib2_descriptor( 5) = 0
8267 grib2_descriptor(iunit) = 103 ! BUG: dovrebbe essere grib2_descriptor(10)
8268 grib2_descriptor(12) = 0
8269 grib2_descriptor(20) = 0      ! Discipline: Meteorological

```

Spiegazione: `iunit` è il numero di unit Fortran del file (41), usato per l'accesso al file stesso. Nella riga del descrittore GRIB2, per un errore di copia-incolla, `iunit` viene usato come indice dell'array `grib2_descriptor` invece dell'indice fisico corretto (quasi certamente 10, che in tutte le altre routine della stessa subroutine corrisponde al *type of level*). Il risultato è che l'elemento 41 riceve il valore 103 (*Height above ground*), mentre l'elemento 10 rimane a 0 — tipo di livello non specificato.

Impatto: I file GRIB2 di riflettività radar hanno il campo “tipo di livello” errato; i decodificatori GRIB2 (`ecCodes`, `wgrib2`) possono rifiutare il file o interpretarlo scorrettamente.

Fix:

```

1 grib2_descriptor(10) = 103 ! Specified height level above ground (m)

```


Bug #7 — llfc non inizializzato in convection_kf

Routine: convection_kf

Codice problematico:

```

11615 do 15 k = 1, nlev
11616   ...
11617   if (p0(k).ge.p300 ) llfc = k      ! llfc = ultimo livello sotto 300 hPa
11618   if (t0(k).gt.t00 ) ml    = k
11619   15 continue
11620   ...
11621   if (lc.gt.llfc) then                ! BUG: llfc mai assegnato se p0 < p300
11622     sempre

```

Spiegazione: llfc viene assegnato solo se $p0(k) \geq p300$ (cioè se il livello è sotto i 300 hPa). Se per qualche ragione la colonna atmosferica non contiene livelli con $p \geq 300$ hPa — situazione atipica ma possibile su domini ad alta quota o in run con colonne molto sottili — llfc rimane con il valore di un'iterazione precedente della coppia (jlon, jlat), producendo un livello LFC artificialmente traslato.

Impatto: Profilo convettivo Kain-Fritsch calcolato su una colonna errata; precipitazione convettiva e campi di umidità/temperatura potenzialmente sbagliati.

Fix: inizializzare llfc a un valore di fallback prima del loop:

```

1 llfc = nlev      ! default: tutto il profilo disponibile
2 do 15 k = 1, nlev
3   ...
4   if (p0(k).ge.p300 ) llfc = k

```

Bug #8 — Divisione per zero nel calcolo dell'umidità relativa del suolo

Routine: programma principale, sezione output punti diagnostici

Codice problematico:

```
2198 write (31,'(100e16.8)') float(jstep)*dtstep/3600., &
2199   qskin(i,j), &
2200   (qg_surf(i,j)-qsoil_min(i,j,1)) / &
2201   (qsoil_max(i,j,1)-qsoil_min(i,j,1)), & ! BUG: /0 se max==min
2202   (qg(i,j,1:nlevg)-qsoil_min(i,j,1:nlevg)) / &
2203   (qsoil_max(i,j,1:nlevg)-qsoil_min(i,j,1:nlevg)) ! BUG: idem
```

Spiegazione: qsoil_max e qsoil_min rappresentano il contenuto d'acqua del suolo a saturazione e alla capacità di campo. Su suoli rocciosi o ghiacciati, o se i dataset fisiografici non sono stati correttamente interpolati, i due valori possono essere identici, annullando il denominatore.

Impatto: FPE o NaN/Inf nell'output diagnostico; può corrompere il file di output del punto.

Fix:

```
1 real :: denom
2 denom = qsoil_max(i,j,1) - qsoil_min(i,j,1)
3 if (denom /= 0.) then
4   write (31,'(100e16.8)') ..., (qg_surf(i,j)-qsoil_min(i,j,1))/denom, ...
5 else
6   write (31,'(100e16.8)') ..., 0., ...
7 endif
```

Bug #9 — Confronto con dimensione errata in raglio

Routine: raglio

Codice problematico:

```
13000 ic = (zrlon-x00)/dlon      ! ic = indice longitudine (direzione X)
13001 jc = (zrlat-y00)/dlat      ! jc = indice latitudine (direzione Y)
13002
13003 if (ic+nx/2.gt.nyg) ic = nxg-nx/2-1 ! BUG: nyg e' la dim. Y, non X
13004 if (jc+ny/2.gt.nyg) jc = nyg-ny/2-1 ! OK: jc direzione Y vs nyg
```

Spiegazione: `ic` è un indice nella direzione X (longitudine), ma il suo vincolo superiore viene confrontato con `nyg` (numero di punti in Y), invece che con `nxg` (numero di punti in X). Su griglie non-quadrate ($nxg \neq nyg$) questo produce una finestra di output traslata o troncata orizzontalmente.

Impatto: Output spray marino centrato sul punto sbagliato quando la griglia ha più righe che colonne (o viceversa).

Fix:

```
1 if (ic+nx/2.gt.nyg) ic = nxg-nx/2-1 ! confronto con nxg
2 if (jc+ny/2.gt.nyg) jc = nyg-ny/2-1
```

Bug #10 — Troncamento intero silente in nfdr

Routine: programma principale, calcolo dimensioni output decimato

Codice problematico:

```
1362 ! nfdr e' un array INTEGER (numero di punti griglia decimata)
1363 nfdr = nldr
1364 nfdr(2) = nldr(2)/float(mhfr) ! BUG: real troncato a integer
1365 nfdr(3) = nldr(3)/float(mhfr) ! BUG: idem
1366 pdr(1) = float(mhfr)*pdr(1) ! OK: pdr e' real
```

Spiegazione: `nfdr` è un array intero che rappresenta il numero di punti della griglia decimata. La divisione `nfdr(2)/float(mhfr)` produce un risultato *real* che viene troncato (non arrotondato) all'intero durante l'assegnazione. Se `nfdr(2)` non è divisibile per `mhfr`, il numero di punti viene sistematicamente sottostimato di 1, causando un mismatch tra le dimensioni dichiarate nell'header del file MHF e i dati effettivamente scritti.

Impatto: Header MHF con numero di punti sbagliato di 1 quando le dimensioni della griglia non sono multipli esatti di `mhfr`. MOLOCH o BOLAM nidificato possono fallire la lettura dei bordi laterali.

Fix: usare la divisione intera con arrotondamento:

```
1 nfdr(2) = (nldr(2) + mhfr/2) / mhfr ! divisione intera arrotondata
2 nfdr(3) = (nldr(3) + mhfr/2) / mhfr
```

oppure, se il troncamento è intenzionale, documentarlo esplicitamente:

```
1 nfdr(2) = int(nldr(2)/float(mhfr)) ! troncamento esplicito
```

Conclusioni

L'analisi ha identificato 10 bug confermati su 15 119 righe di codice. La distribuzione per area funzionale è la seguente:

Area	Critici	Alti	Medi
Interpolazione input (inizializzazione)	1	1	—
MPI/Parallelizzazione	—	1	—
Parametrizzazione fisica (surflayer)	—	1	—
Parametrizzazione fisica (ccloudt)	—	1	—
I/O GRIB2 (radar)	—	1	—
Convezione (Kain-Fritsch)	—	1	—
Output diagnostico punti	—	1	—
Geometria griglia (raglio)	—	—	1
Calcolo dimensioni output	—	—	1
Totale	1	7	2

Priorità di intervento suggerita:

1. **Bug #1 e #2** (righe 1381/1383) — da correggere immediatamente: ogni run con input COSMO è affetto da condizioni iniziali errate.
2. **Bug #3** (riga 3387) — rilevante in tutti i run paralleli con maschere intere.
3. **Bug #5 e #7** (righe 5903, 11627) — condizioni rare ma non impossibili in situazioni meteorologiche estreme.
4. **Bug #6** (riga 8336) — rilevante solo con output radar attivo.
5. **Bug #4 e #8** (righe 5028, 2200) — condizioni di bordo che dipendono dai dati di input.
6. **Bug #9 e #10** (righe 13002, 1364) — impatto limitato a griglie non-quadrate e dimensioni non divisibili.